

A Double-Entry Ledger

A worked example in Literate Driven Development

Built with weft, gated by ldd

September 24, 2026

Contents

1	The domain: money that must always balance	2
1.1	The problem	2
1.2	Vocabulary	2
2	Architecture	3
2.1	Concepts	3
2.2	Technology	3
2.3	Decision records	3
2.3.1	Decision: integer cents, never floating point	3
2.3.2	Decision: the invariant lives in Ledger, not in Accounts	3
2.3.3	Decision: transactions are atomic	4
2.4	The dependency contract	4
3	Roadmap	5
3.1	Stage 1 — the walking skeleton	5
3.2	Later stages (planned)	5
3.3	The concept ledger	5
3.4	Seam contracts	5
4	Accounts	7
4.1	The public seam	7
4.2	The store	7
4.3	Seed: the demo accounts	8
5	Ledger	10
5.1	The public seam	10
5.2	Enforcing the invariant	10
5.3	Seed: the demo posting	11
6	Api	12
6.1	The walking skeleton	12
7	Assembly	14

Chapter 1

The domain: money that must always balance

A *ledger* records movements of value between accounts. Its one inviolable law is *double entry*: every transaction posts equal and opposite amounts, so the sum of all balances never changes. A system that can violate this law is not a ledger with a bug — it is not a ledger.

1.1 The problem

We want the smallest honest core of a ledger: open accounts, post transactions between them, read balances — with the double-entry invariant enforced *at write time*, so an unbalanced transaction cannot be committed at all. Nightly reconciliation against a spreadsheet is exactly the failure mode we refuse: by the time a mismatch is found, wrong numbers have already been read and acted upon. The invariant belongs in the code that writes, not in a report that reads.

1.2 Vocabulary

Every term below becomes part of the system’s language — the *domain* prefix of a chunk name, and an entry in the glossary. If a later chunk name uses a word, that word is defined here.

Account a named store of value. Its **balance** is an integer number of **cents**; we never use floating point, because in a ledger rounding is not cosmetic but a correctness failure.

Posting a single movement of a signed amount (in cents) into one account. A positive amount is a debit to that account’s balance, a negative one a credit.

Transaction an ordered set of postings that **must sum to zero**. This is double entry stated arithmetically.

Invariant the property the system guarantees at all times: the sum of every account balance equals zero, because it started at zero and only balanced transactions were ever applied.

Chapter 2

Architecture

2.1 Concepts

The system decomposes into three concepts, each a chapter that crosses the whole stack (types, logic, and the seams it exposes to its neighbours):

- **Accounts** — the store of accounts and their balances. It owns what an account *is* and the only operations that change a balance.
- **Ledger** — transactions and postings. It owns the *invariant*: it refuses to apply a transaction whose postings do not sum to zero.
- **Api** — the surface. For this stage it is a small command line program that opens accounts, posts one transaction, and prints the balances.

2.2 Technology

Plain C, one static binary, no dependencies beyond libc. The domain is small and correctness-critical, which is exactly where C's directness helps and its footguns (floating point, partial mutation) must be designed out rather than hoped away.

2.3 Decision records

2.3.1 Decision: integer cents, never floating point

Chosen: balances are `long` cents. Considered and rejected: `double` dollars, because $0.1 + 0.2 \neq 0.3$ in binary floating point, and a ledger that cannot represent a dime exactly is not fit for purpose.

2.3.2 Decision: the invariant lives in Ledger, not in Accounts

Chosen: `Accounts` exposes a neutral `acc_apply(id, delta)` that changes one balance, and `Ledger` is the only place that knows a transaction must sum to zero. Considered and rejected:

enforcing balance in Accounts, which would force every balance change to know about transactions and couple the two concepts permanently. Keeping the invariant in one concept is what makes it auditable.

2.3.3 Decision: transactions are atomic

Chosen: `ledger_post` validates the sum *and* every account before it mutates anything, so a rejected transaction leaves no partial effect. This is why Accounts exposes an `acc_exists` check separate from `acc_apply`.

2.4 The dependency contract

A concept may reference another only along an allowed edge. Exactly three are allowed here: **Ledger** $\rightarrow$ **Accounts**, **Api** $\rightarrow$ **Ledger** and **Api** $\rightarrow$ **Accounts**. Every other direction is forbidden, and the forbidden ones are the design: **Accounts must not know about Ledger** (the store is ignorant of transactions), and **no concept below Api may depend on Api** (the domain must not reach up into its surface).

Reviewing this is a reading task, not a tool task: `weft -e NAME --callers` shows who reaches into a chunk, and a reference that crosses one of the forbidden directions is a defect even when it compiles.

Chapter 3

Roadmap

3.1 Stage 1 — the walking skeleton

The thinnest slice that runs end to end and proves the architecture: open two accounts, post one balanced transaction between them, print both balances, and show that an *unbalanced* transaction is refused. It touches every concept thinly — Accounts (open, apply, balance), Ledger (post, invariant), Api (the demo main) — so when it builds green the shape is proven and everything after is thickening.

3.2 Later stages (planned)

- **Stage 2 — Reporting:** a trial balance and per-account statements over the ledger. New concept **Reporting**, depending on Ledger and Accounts.
- **Stage 3 — Persistence:** an append-only journal on disk so balances survive a restart.

3.3 The concept ledger

The single source of truth for what exists and what is planned. Lifecycle: **planned** → **stubbed** → **implemented** → **stable**.

Concept	Status	Depends on	Stage	Chapter
Accounts	implemented	—	1	4
Ledger	implemented	Accounts	1	5
Api	implemented	Ledger, Accounts	1	6
Reporting	planned	Ledger, Accounts	2	—

3.4 Seam contracts

The chunks at the boundaries between concepts, named so no one improvises an interface. In Stage 1 they are filled immediately by the concept chapters:

Accounts: `header` the account type and the balance-changing operations every other concept is allowed to call.

Ledger: `header` the `Entry` type and `ledger_post`, the only door through which balances change in a transaction.

Accounts: `demo accounts` / **Ledger:** `demo posting` seed snippets each concept owns and `Api` composes into the walking skeleton.

Chapter 4

Accounts

The Accounts concept owns what an account *is* and the only operations that move a balance. It knows nothing about transactions — that ignorance is deliberate (see the architecture chapter): the store of value must not depend on the rules that govern movement, or the two could never be reasoned about apart.

4.1 The public seam

Everything another concept may call lives in one header. An account is opened by name and referred to afterwards by a small integer id. Balances are signed `long` cents. Four operations are exposed: open an account, read a balance, apply a signed delta, and — so a caller can validate before it mutates — test whether an id refers to a live account.

```
<Accounts: header 7> ≡
#ifndef LEDGER_ACCOUNTS_H
#define LEDGER_ACCOUNTS_H

#define ACC_MAX 64                /* the store holds at most this many accounts */

int      acc_open(const char *name); /* -> id >= 0, or -1 if the store is full */
long     acc_balance(int id);        /* balance in cents, 0 for an unknown id */
int      acc_apply(int id, long delta); /* add delta cents; 0 ok, -1 bad id */
int      acc_exists(int id);         /* 1 if id is a live account, else 0 */
const char *acc_name(int id);        /* the account's name, or "?" */

#endif
□
7 · used in 14a
```

4.2 The store

The implementation is a fixed array of accounts. A real ledger would persist to a journal (Stage 3); the walking skeleton keeps the store in memory, which is enough to prove the invariant. Note

that `acc_apply` is neutral: it adds a delta and asks no questions about balance. The rule that deltas must come in balanced sets belongs to Ledger, not here.

```
< Accounts: implementation 8 > ≡
#include "accounts.h"
#include <string.h>

static struct { char name[32]; long balance; int used; } accounts[ACC_MAX];

int acc_open(const char *name) {
    for (int i = 0; i < ACC_MAX; i++) {
        if (!accounts[i].used) {
            accounts[i].used = 1;
            accounts[i].balance = 0;
            strncpy(accounts[i].name, name, sizeof accounts[i].name - 1);
            accounts[i].name[sizeof accounts[i].name - 1] = '\0';
            return i;
        }
    }
    return -1;
}

int acc_exists(int id) {
    return id >= 0 && id < ACC_MAX && accounts[id].used;
}

long acc_balance(int id) {
    return acc_exists(id) ? accounts[id].balance : 0;
}

int acc_apply(int id, long delta) {
    if (!acc_exists(id)) return -1;
    accounts[id].balance += delta;
    return 0;
}

const char *acc_name(int id) {
    return acc_exists(id) ? accounts[id].name : "?";
}
□
```

8 · used in 14b

4.3 Seed: the demo accounts

The walking skeleton needs two accounts to move money between. Opening them is an Accounts concern, so the snippet lives here and `Api` composes it. When this chunk is expanded inside `main`, it declares `cash` and `revenue` in that scope.

```
< Accounts: demo accounts 9 > ≡
int cash    = acc_open("cash");
int revenue = acc_open("revenue");
```

□

9 · used in 12

Chapter 5

Ledger

The Ledger concept owns the one law that makes this a ledger: a transaction is a set of postings that sum to zero, and nothing else may be committed. It is the only place in the system that knows this rule, which is what makes the rule auditable — to check that money is conserved, a reader looks in exactly one function.

5.1 The public seam

A **Entry** is one posting: a signed amount, in cents, into one account. A transaction is an array of them. `ledger_post` is the only door through which balances change as a group.

```
< Ledger: header 10 > ≡
#ifndef LEDGER_LEDGER_H
#define LEDGER_LEDGER_H

typedef struct { int account; long amount; } Entry; /* one posting, cents */

/* Post a balanced transaction. Returns 0 on success; -1 if the entries do
   not sum to zero (the invariant) or name an account that does not exist.
   On failure NO account is modified --- the post is atomic. */
int ledger_post(const Entry *entries, int n);

#endif
□
10 · used in 14c
```

5.2 Enforcing the invariant

The implementation makes the atomicity promise real by validating everything before it changes anything: first the postings must sum to zero, then every referenced account must exist. Only once both checks pass does it apply the deltas through `acc_apply`. Because the store was validated up front, the apply loop cannot fail partway and leave a half-posted transaction. This is the single allowed dependency of Ledger on Accounts.

⟨ Ledger: implementation 11a ⟩ ≡

```
#include "ledger.h"
#include "accounts.h"

int ledger_post(const Entry *entries, int n) {
    long sum = 0;
    for (int i = 0; i < n; i++) sum += entries[i].amount;
    if (sum != 0) return -1; /* the invariant: money is conserved */

    for (int i = 0; i < n; i++)
        if (!acc_exists(entries[i].account)) return -1; /* validate before mutate */

    for (int i = 0; i < n; i++)
        acc_apply(entries[i].account, entries[i].amount); /* now safe: all valid */
    return 0;
}
```

□

11a · used in 14d

5.3 Seed: the demo posting

The walking skeleton's one transaction moves one hundred dollars: a debit to **cash** and an equal credit to **revenue**. The two postings sum to zero, so the ledger accepts it. Building the transaction is a Ledger concern, so the snippet lives here; Api composes it after the demo accounts exist, and it reads the **cash** and **revenue** ids from that scope.

⟨ Ledger: demo posting 11b ⟩ ≡

```
Entry txn[] = { { cash, +10000 }, { revenue, -10000 } };
int rc = ledger_post(txn, 2);
```

□

11b · used in 12

Chapter 6

Api

The Api concept is the surface a person touches. For the walking skeleton it is a single `main` that tells the whole story: open the demo accounts, post the demo transaction, print the balances, and then — the proof that the invariant is real — try to post an *unbalanced* transaction and show it is refused. Api depends on both other concepts; nothing depends on Api.

6.1 The walking skeleton

`main` composes the two seed snippets its neighbours own, then reports. After the balanced posting, `cash` holds \$100.00 and `revenue` holds $-\$100.00$; their sum is zero, which is the invariant made visible on the last line. The deliberately unbalanced transaction (a lone \$5.00 debit) must be rejected — if it were ever accepted, the final sum would drift from zero and the program would say so.

```
< Api: main 12 > ≡
#include <stdio.h>
#include "accounts.h"
#include "ledger.h"

int main(void) {
    < Accounts: demo accounts 9 >
    < Ledger: demo posting 11b >

    printf("posted transaction: %s\n", rc == 0 ? "OK" : "REJECTED");
    printf("  %-8s %8.2f\n", acc_name(cash),    acc_balance(cash)    / 100.0);
    printf("  %-8s %8.2f\n", acc_name(revenue), acc_balance(revenue) / 100.0);

    /* The invariant, demonstrated: an unbalanced transaction is refused. */
    Entry bad[] = { { cash, +500 } };          /* sums to 500, not zero */
    printf("unbalanced transaction: %s\n",
           ledger_post(bad, 1) == 0 ? "ACCEPTED (bug!)" : "REJECTED (correct)");

    long total = acc_balance(cash) + acc_balance(revenue);
    printf("sum of all balances: %ld cents (must be 0)\n", total);
    return 0;
}
```

}

□

12 · used in 15

Chapter 7

Assembly

This is the only file with `@o` directives. The concept chapters never name an output path; they define chunks, and the assembly routes those chunks to the layer-oriented files the C toolchain expects. Because routing lives here alone, the physical layout can change — split a file, rename a directory — without touching a single concept narrative.

The store and its header:

```
⟨ src/accounts.h 14a ⟩ ≡  
⟨ Accounts: header 7 ⟩
```

□

14a

```
⟨ src/accounts.c 14b ⟩ ≡  
⟨ Accounts: implementation 8 ⟩
```

□

14b

The ledger and its header:

```
⟨ src/ledger.h 14c ⟩ ≡  
⟨ Ledger: header 10 ⟩
```

□

14c

```
⟨ src/ledger.c 14d ⟩ ≡  
⟨ Ledger: implementation 11a ⟩
```

□

14d

The command-line surface:

$\langle \text{src/main.c } 15 \rangle \equiv$

$\langle \text{Api: main } 12 \rangle$

□

15